CLOUD NATIVE ECOSYSTEM / KUBERNETES / PLATFORM ENGINEERING

# Проблемы облачных нативных приложений: установка «ходячего скелета»

Узнайте, как начать работу с шаблонизаторами и менеджерами пакетов, чтобы упростить развертывание и настройку в больших масштабах.

13 мая 2026 года, 9:00 Авторы издательства Manning Book

Hartono Creative Studio для Unsplash+



**Примечание редактора:** Эта статья является выдержкой из главы 1 книги издательства Manning *«Разработка платформы на Kubernetes»*. В этой выдержке основное внимание уделяется управлению ресурсами Kubernetes, определяемыми с помощью YAML, — развертываниями, сервисами, Ingress и ConfigMaps/секретами — в условиях масштабирования. Чтобы узнать о последних передовых методах работы с Kubernetes и решениях с открытым исходным кодом от сообщества Kubernetes, скачайте книгу целиком.

нативными приложениями на примере развертывания микросервисного приложения в локальном кластере Kubernetes с использованием Kubernetes в Docker. Теперь пришло время развернуть наш «ходячий скелет».

Чтобы запускать контейнерные приложения поверх Kubernetes, вам нужно упаковать каждую из служб в виде образа контейнера, а также определить, как эти контейнеры будут настроены для работы в вашем кластере Kubernetes.

## Определение сервисов и ресурсов в Kubernetes с помощью YAML

Для этого Kubernetes позволяет определять различные типы ресурсов (в формате YAML), чтобы настраивать способы работы контейнеров и их взаимодействие друг с другом. Наиболее распространенные типы ресурсов:

- *Развертывания*: декларативно определяйте, сколько реплик вашего контейнера должно быть доступно для корректной работы приложения. Развертывания также позволяют выбрать, какой контейнер (или контейнеры) мы хотим запустить и как эти контейнеры должны быть настроены (с помощью переменных среды). Запуск наших облачных приложений.
- *Сервисы*: декларативно определяйте высокоуровневую абстракцию для маршрутизации трафика к контейнерам, созданным в рамках ваших развертываний. Она также выступает в роли балансировщика нагрузки между репликами внутри ваших развертываний. Сервисы позволяют другим сервисам и приложениям внутри кластера использовать для связи имя сервиса вместо физического IP-адреса контейнеров, обеспечивая так называемое обнаружение сервисов.
- *Ingress*: декларативное определение маршрута для перенаправления трафика из-за пределов кластера к сервисам внутри кластера. С помощью определений Ingress мы можем предоставлять доступ к сервисам, которые требуются клиентским приложениям, работающим за пределами кластера.
- *ConfigMap/secrets*: декларативное определение и хранение объектов configuration для настройки экземпляров наших сервисов. Секретная информация — это конфиденциальные данные, доступ к которым должен быть защищен.

## Why YAML alone becomes hard to manage

se YAML files will be complex and hard to manage if you have large applications with tens or hundreds of services. Keeping track of the changes and

resources, and other resources are available such as the official Kubernetes documentation page.



In this book [which can be downloaded here], we will concentrate on how to deal with these resources for large applications and the tools that can help us with that task. The following section provides an overview of the tools to package and install components into your Kubernetes cluster.

## Packaging and installing Kubernetes applications

There are different tools to package and manage your Kubernetes applications.
st of the time, we can separate these tools into two main categories: templating

TRENDING STORIES

1. How we cut AI costs by 80%
2. In 2026, AI Is Merging With Platform Engineering. Are You Ready?
3. The DIY platform trap that's burning out engineering teams
4. Platform Engineering vs. DevOps Misses the Point
5. Paved Roads, Golden Paths, Guardrails and Railroads

## What are templating engines in Kubernetes?

Let's discuss these two kinds of tools: why would you need a templating engine? What kind of packages do you want to manage? A templating engine allows you to reuse the same resource definitions in different environments where applications might require slightly different parameters.

The textbook example of the need to template your resources is database URLs. If your service needs to connect to different database instances in different environments, such as the testing database in the testing environment and the production database in the production environment, you want to avoid maintaining two copies of the same YAML file but with different URLs.

The figure below shows how you can now add variables to the YAML files, and the engine will then find and replace these variables with different values depending on where you want to use the final (rendered) resource.

---

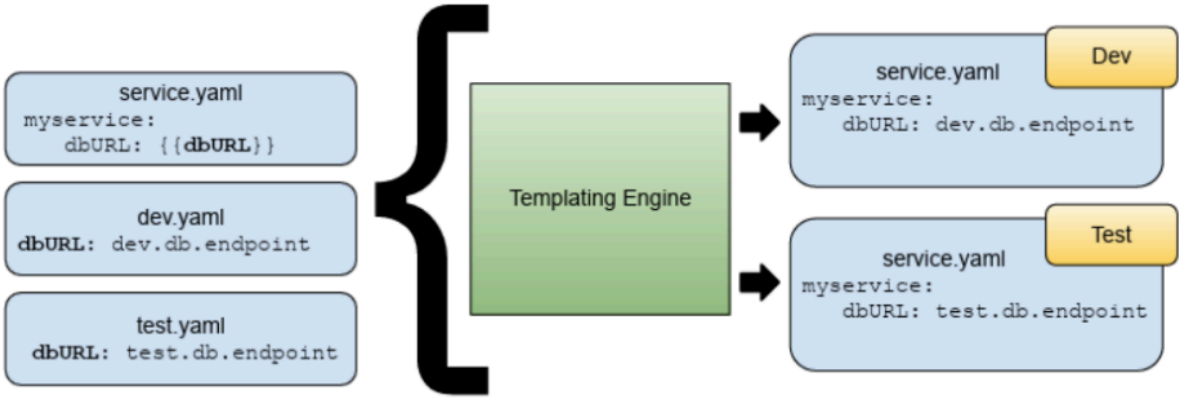
"Using a templating engine can save you a lot of time maintaining different copies of the same file, because when files start to pile up, maintaining them becomes a fulltime job."

---

Using a templating engine can save you a lot of time maintaining different copies of same file, because when files start to pile up, maintaining them becomes a fulltime job. There are several tools in the community to deal with templating

check out are:

- *Kustomize*: https://kustomize.io/




- *Carvel YTT*
- *Helm Templates*

## What are package managers for Kubernetes?

Now, what do you do with all these files?

It is quite a natural urge to organize these files in logical packages. If you are building an application that is composed of different services, it might make sense to group all the resources related to a service inside the same directory or even in the same repository that contains the source code for that service.

You also want to make sure that you can distribute these files to the teams deploying these services to different environments, and you quickly realize that you need to version these files in some way. This versioning might be related to the version of your service itself or with a high-level logical aggregation that makes sense for your application.

---

"When we talk about grouping, versioning, and distributing these resources, we are describing the responsibility of a package manager."

---

teams are already used to working with package managers no matter the technology stack they use. Maven/Gradle for Java, NPM for NodeJS, APT-GET for Linux/Debian/ Ubuntu packages, and more recently, containers and container registries for cloudnative applications.

## Core responsibilities of a Kubernetes package manager

So, what does a package manager for YAML files look like? What are the package manager's main responsibilities? As a user, a package manager allows you to browse available packages and their metadata to decide which package you want to install. Once you have decided which package you want to use, you should be able to download it and then install it.

Once the package is installed, you would expect, as a user, to be able to upgrade to a newer version of the package when it becomes available. Upgrading/updating a package requires manual intervention, meaning that as a user, you will explicitly tell the package manager to upgrade the installation of a certain package to a newer (or latest) version.

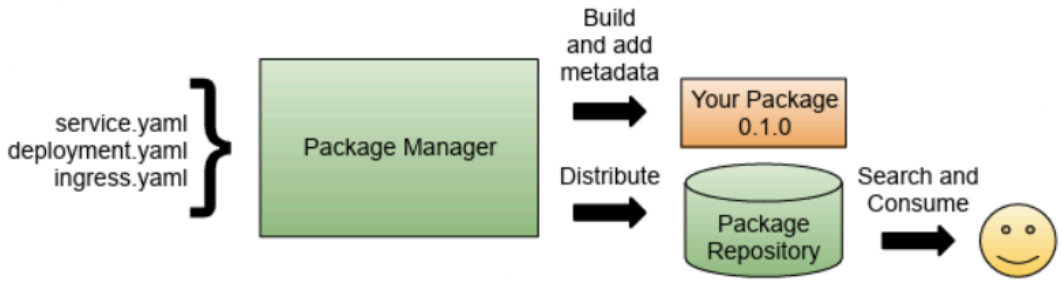
From a package provider's point of view, a package manager should offer a convention and structure to create packages and a tool to package the files you want to distribute. Package managers deal with versions and dependencies, meaning that if you create a package, you must associate a version number with it.

Some package managers use the semver (semantic versioning) approach, which uses three numbers to describe the package maturity (1.0.1, where these numbers represent the major, minor, and patch versions).

A package manager doesn't need to provide a centralized package repository, but they often do. This package repository is in charge of hosting packages for users to consume.

Central repositories are useful because they provide access to developers with thousands of packages ready to be used. Some examples of these central repositories are Maven Central, NPM, Docker Hub, GitHub Container Registry, etc. These repositories are in charge of indexing the package's metadata (which can include versions, labels, dependencies and short descriptions) to make them ›rchable by users.

repository is to allow package producers to upload packages and package consumers to download packages from them (see below).




## Popular Kubernetes tools: Helm, Imgpkg, Kapp, Terraform

When we talk about Kubernetes, Helm is a very popular tool that provides both a package manager and a templating engine. But there are others worth looking into, such as:

- Imgpkg , which uses Container registries to store the packages.
- Kapp , which provides higher-level abstractions to group resources as applications.
- Tools like Terraform and Pulumi that allow you to manage infrastructure as code.

In the following section, we will look at using Helm to install the Conference application into our Kubernetes cluster. Download the entire book to keep reading.

## Frequently asked questions

### 1. What is a walking skeleton in cloud native applications?

A walking skeleton is a minimal deployment that demonstrates basic architecture and connectivity between microservices. It serves as a foundation for testing your deployment pipeline and Kubernetes configuration before building full functionality.

### 2. Why do you need templating engines for Kubernetes YAML files?

Templating engines solve the problem of maintaining multiple YAML copies across different environments. They let you use variables that get replaced with environment-specific values like database URLs, saving time when managing complex applications.

Templating engines handle configuration management by replacing variables for different environments. Package managers focus on grouping, versioning, and distributing collections of Kubernetes resources as logical packages.

### 4. How do you manage Kubernetes applications with multiple services at scale?

Use the four core Kubernetes resources (Deployments, Services, Ingress, ConfigMaps/Secrets) combined with both templating engines and package managers. Tools like Helm provide both capabilities to avoid manually managing hundreds of YAML files.

TNS

---


This is an excerpt from the Manning Early Access Program (MEAP) book "Effective Platform Engineering" by Ajay Chankramath, MD & CTO of Industry Solutions, Platform and Product Engineering for Brillio; Nic Cheneweth, Principal Consultant at Thoughtworks; Bryan Oliver, Principal Consultant...

Read more from Manning Book Authors →



---



---

TRENDING STORIES

1. How we cut AI costs by 80%
2. In 2026, AI Is Merging With Platform Engineering. Are You Ready?
3. The DIY platform trap that's burning out engineering teams
4. Platform Engineering vs. DevOps Misses the Point
5. Paved Roads, Golden Paths, Guardrails and Railroads

# Receive a free roundup of the most recent TNS articles in your inbox each day.

svo1975pnv1982@gmail.com

SUBSCRIBE

Kubernetes
Observability
Operations
Platform Engineering

Newsletter
TNS RSS Feeds

THE NEW STACK

About / Contact
Sponsors
Advertise With Us
Contributions

roadmap.sh

Community created roadmaps, articles, resources and journeys for developers to help you choose your path and grow in your career.

Frontend Developer Roadmap
Backend Developer Roadmap
Devops Roadmap

FOLLOW TNS



Disclosures
Terms of Use
Advertising Terms & Conditions
Privacy Policy
Cookie Policy